

Smart Calorie Scale

By: Charlie Han, Aiden Odame, Mariah Gomez, Miechilly Dong



What is a Food Scale?

- **The Basic Tool:**
  - A **food scale** is a device used to measure the *mass* (weight) of food items (e.g., grams or ounces).
  - Unlike measuring cups (which measure volume), a food scale measures **exact weight**, leading to more consistent cooking and accurate dietary tracking.
- **Why use one?**
  - **Baking:** 100g of flour is always 100g, but 1 cup of flour can vary wildly.
  - **Dieting:** You cannot guess portion sizes accurately; a scale removes the guesswork.
- **Our Project:** We took this basic tool and added "intelligence" (Smart) so it tells you **calories**, not just weight.



Project Overview

What is the Smart Calorie Scale?

- A smart food scale that accurately weighs food items and instantly calculates their estimated calorie content.

- **How Does It Work?**
  - **Step 1:** User places food item on the scale.
  - **Step 2:** Arduino load cell sensor captures the weight.
  - **Step 3:** MATLAB processes the weight and converts it into grams.
  - **Step 4:** MATLAB displays the final calorie count to the user.
- **Key Technologies Used:**
  - **Arduino:** Hardware sensor input (the “hands”).
  - **MATLAB:** Weight processing, math, and user interface (the "brain").

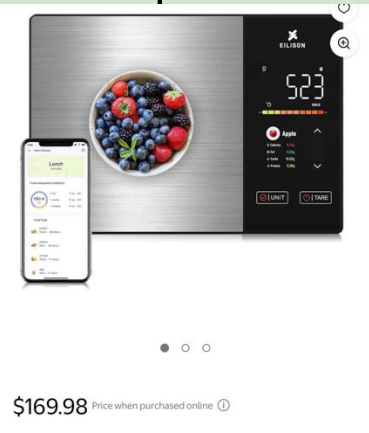
Problem & Motivation

What Problem Are We Solving?

- Tedious: Manually tracking calories requires looking up every food item and measuring portions.
- Error-prone: People often rely on rough estimations or inaccurate serving sizes ("a handful").
- Expensive alternatives: Existing smart scales cost $50-$150+ and often lock you into their specific app.

Why We Chose This Project:

- To create a **cost-effective** solution using basic hardware.
- To combine **hardware (Arduino)** with **software (MATLAB)** into a complete functional system.



Goals & Responsibilities



- **Project Goal:**
- Build a functional prototype where a user places food on the scale and receives an accurate calorie estimate within seconds.
- **Strategic Plan:**
- 1. Hardware Setup: Assemble Arduino + Load Cell.
- 2. Data Acquisition: Read sensor data into MATLAB.
- 3. Weight Processing: Convert raw voltage to grams.
- 4. Calorie Calculation: Apply nutritional math (Grams × Calories/g).
- 5. Display: Show result to user.
- **Team Roles:**
- Mariah Gomez (PM): Timeline, coordination, final report.
- Charlie Han (MATLAB): Weight conversion, interface, display.
- Miechilly Duong (Hardware): Arduino setup, load cell calibration, raw data transfer.
- Aiden Odame (Design): Presentation, planner, coordination.

How We Calculate Calories

The Formula:

- Calories = Weight (grams) × Calories per gram

Example: 100g of Ground Beef

- 90% Lean (10% Fat) = 170 calories
- 70% Lean (30% Fat) = 350 calories
- Same weight. Different calories.

Our MATLAB Code:

- Measures weight in grams
- Multiplies by calories per gram for that food
- Displays total calories

```
T = readtable('Data.txt');
avgVar2 = mean(T.Var2);
myInput = input("What type of food are you weighing?\na.Chicken Breast\nb.Ground Beef\nc.Porkbelly\n",'s');
if strcmpi(myInput,'a')
    fprintf("Chicken Breast\n")
    calories = avgVar2 * 165/100;
    protein = avgVar2 * 31/100;
    fat = avgVar2 * 3.6/100;
    carbs = 0;
    fprintf('For %.1f g of chicken breast,\n', avgVar2);
    fprintf('  Calories: %.2f kcal\n  Protein:  %.2f g\n  Fat:      %.2f g\n  Carbs:    %.2f g\n', calories, protein, fat, carbs);
elseif strcmpi(myInput,'b')
    fprintf("Ground Beef\n")
    fprintf("What is the lean/fat ratio?\n")
    leanPercent = input("Lean%?\n");
    fatPercent = input("Fat%?\n");
    if leanPercent + fatPercent == 100;
    calories = avgVar2 * leanPercent/100 * 131/100 + 9 * avgVar2 * fatPercent/100;
    protein = avgVar2 * leanPercent/100 * 25/100 + 0 * avgVar2 * fatPercent/100;
    fat = avgVar2 * fatPercent/100;
    carbs = 0;
    fprintf('For %.1f g of ground beef, (Lean: %.1f%%, Fat: %.1f%%):\n', avgVar2, leanPercent, fatPercent);
    fprintf('  Calories: %.2f kcal\n  Protein:  %.2f g\n  Fat:      %.2f g\n  Carbs:    %.2f g\n', calories, protein, fat, carbs);
    else
        error("Invalid input. Percentages must add up to 100%");
    end
elseif strcmpi(myInput,'c')
    fprintf("Porkbelly\n")
    calories = avgVar2 * 520/100;
    protein = avgVar2 * 10/100;
    fat = avgVar2 * 51/100;
    carbs = 0;
    fprintf('For %.1f g of Porkbelly,\n', avgVar2);
    fprintf('  Calories: %.2f kcal\n  Protein:  %.2f g\n  Fat:      %.2f g\n  Carbs:    %.2f g\n', calories, protein, fat, carbs);
else
   error('Invalid input. Please enter a, b, or c.');
end
```

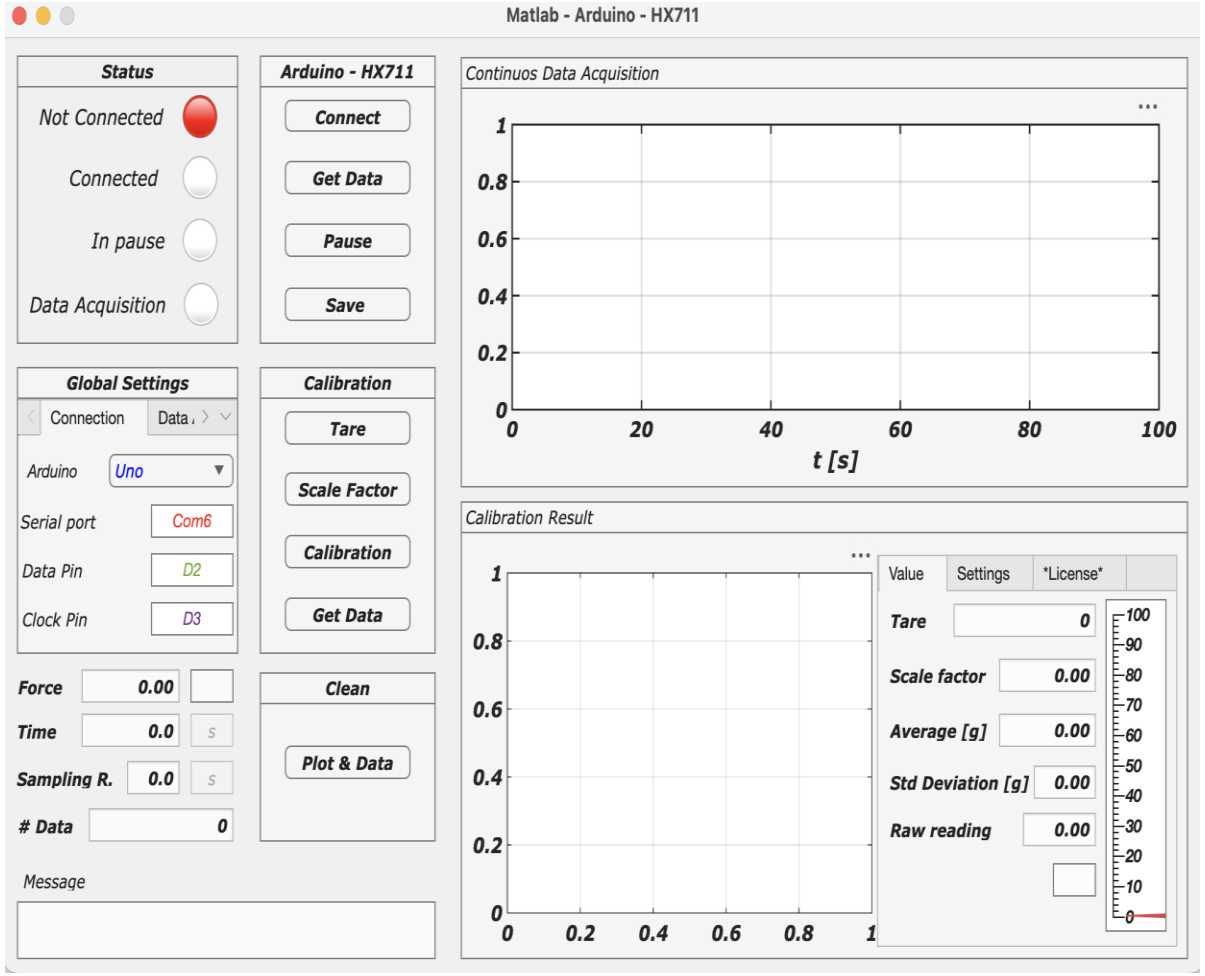
Software & MATLAB Implementation

- Arduino Library:

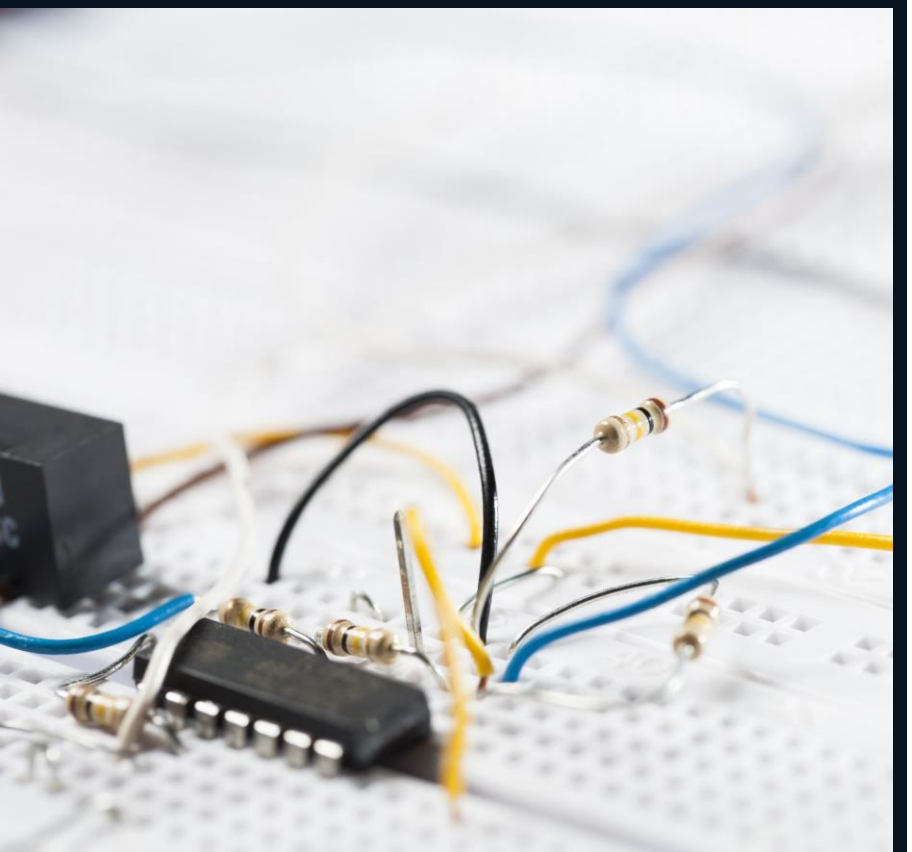
Used the "Basic Custom Arduino Library for HX711" by Nicholas Giaacoboni to read the load cell.

MATLAB Interface Features:

- Connection Control: Connect/Disconnect to Arduino via Serial Port.
- Calibration Tools: Tare (zeroing the scale) and Scale Factor adjustment.
- Real-time Plotting: Visualizing force (grams) vs. Time (seconds).



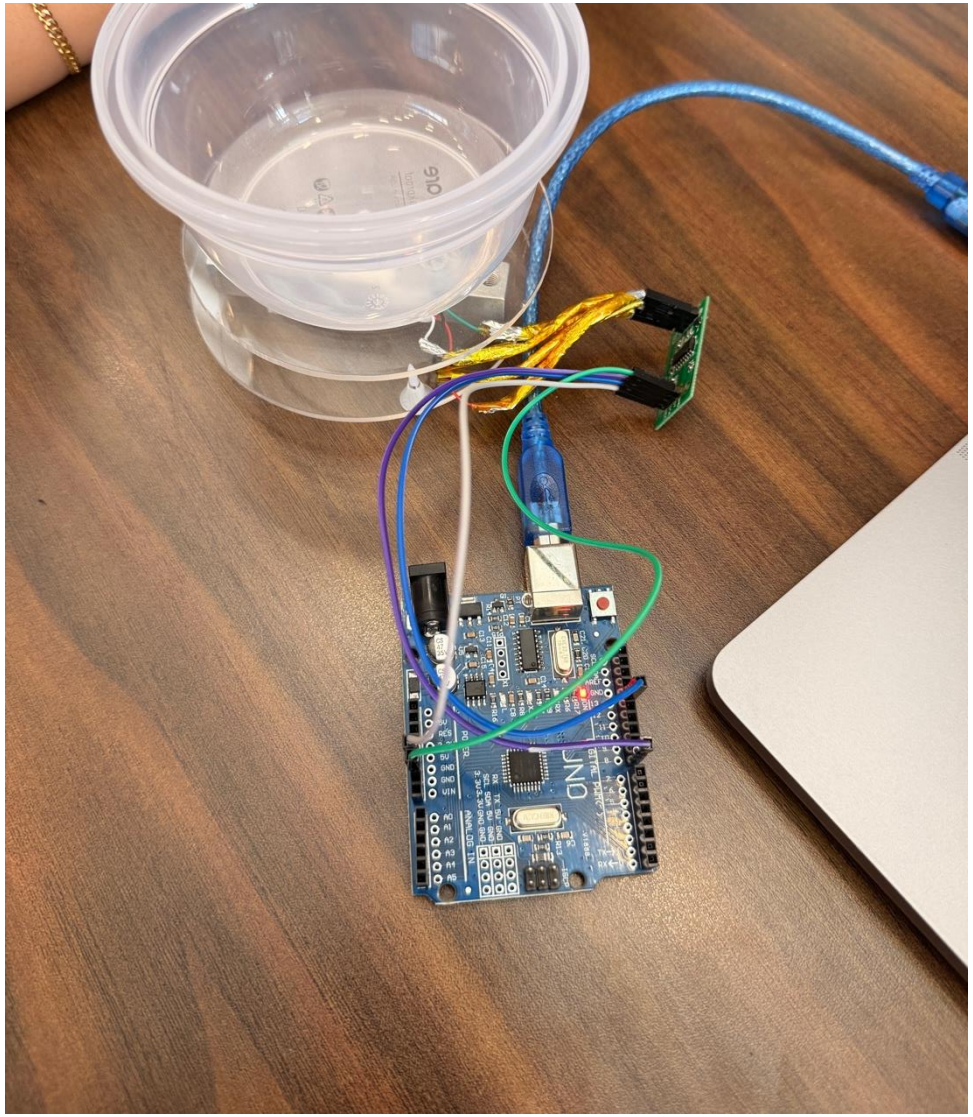
Challenges & Solutions



- **Main Challenge:**
- Accuracy of the Arduino system
- **Specific Limitations:**
- Uneven surfaces: The load cell requires a rigid, flat base
- Poor wiring: Loose connections caused signal noise (jumpy readings)
- Over-sensitive load cell: The sensor picks up tiny vibrations (breathing, table bumps)
- **How we addressed it:**
- Improved wiring connections (soldered joints)
- Order a new load cell sensor

Results

- **What Works:**
- Scale successfully captures weight via load cell
- MATLAB converts raw readings to grams
- Calorie calculation returns correct output
- **Live Demo:**
- We will show the scale weighing a real object
- MATLAB screen will display the calorie count in real time



Conclusion

- **What We Learned:**
  - Real-world sensors are fragile
  - Integrating Arduino with MATLAB requires strict timing and calibration.
- **Did we meet the goal?**
  - **Yes.** We created a functional prototype that maps weight to a calorie estimate.
- **Limitations:**
  - Currently requires manual entry of "calories per gram" (does not automatically recognize the *type* of food, e.g., apple vs. cookie).
  - Load cell drift over time.
- **Future Improvements:**
  - Add a feature to track total calories through different runs of the program.
  - Implement more foods and be able to look up the macros off the internet



Citations

- NICHOLAS GIACOBONI (2026). Basic Custom Arduino Library for HX711(https://www.mathworks.com/matlabcentral/fileexchange/66641-basic-custom-arduino-library-for-hx711), MATLAB Central File Exchange. Retrieved April 2, 2026.
- “Arduino.” *Www.mathworks.com*, www.mathworks.com/hardware-support/arduino.html.
- https://www.amazon.com/Geekstory-Converter-Breakout-Electronic-Raspberry/dp/B0B6NRW8GG/ref=sr_1_8?crid=2UQYXJB1X9GAH&dib=eyJ2IjoiMSJ9.WN5KRTKR35IEtukubJe6j3zfF1vUSKTQB3iMP5r-u7BUEGmVTcnjcS4kTtfbGOnGyuJBG_Ul6KLcqbYvwm8Ua_lYU3L3RGx2W-Vibn3YOELZLdak-7FMk_mXOAzzpDIkJ5VbtAElajVSgAE4OZKmKP5E86cHyBizUaUloP9mSxLVR3ZqVT3cdBTVIDkV3vyOB_l6U50ZEpLOC6xTv7VdYzLxNs8zDZXSXzJoeKc1Zw4.HRo0Q5GvDF7QbZqIsVNockGCpzHGq9hzn7oppnbfr9o&dib_tag=se&keywords=arduino+scale+kit&qid=1776970464&sprefix=arduino+scale+kit%2Caps%2C213&sr=8-8